



Reporte de Ejecución de Skills

Análisis de Buenas Prácticas: Backend y Frontend

Integrantes (Programangos):

Carlos Daniel Albarracín Cruz

Pablo Andrés Pérez Beltrán

David Nickolai Parra Ariza

Jhoan Sebastián Yaya García

Docente: Oscar Eduardo Álvarez Rodríguez

Fecha: 14/06/2026

Curso:

2016701: Ingeniería de Software I

Facultad de Ingeniería, Universidad Nacional de Colombia

1. Introducción

El presente reporte documenta la **ejecucion de las skills** de revisión de buenas prácticas definidas para el proyecto SISA (Sistema de Supervivencia Académica).

Se definieron dos skills en formato Markdown: **skill_back.md** para el backend (Django + DRF + PostgreSQL) y **skill_front.md** para el frontend (Vue 3 + TypeScript + Pinia + Tailwind). Cada skill fue ejecutada por un agente de inteligencia artificial (**OpenCode**) sobre el código real del proyecto, el cual recorrió los archivos de cada capa y generó un reporte de cumplimiento estructurado.

Cada skill evalúa tres dimensiones: buenas prácticas del lenguaje (convenciones de nombres, clean code), buenas prácticas de arquitectura (separación de capas, dependencias entre módulos) y criterios de documentación y configuración del proyecto. Los criterios están clasificados en MUST (obligatorio) y SHOULD (recomendado).

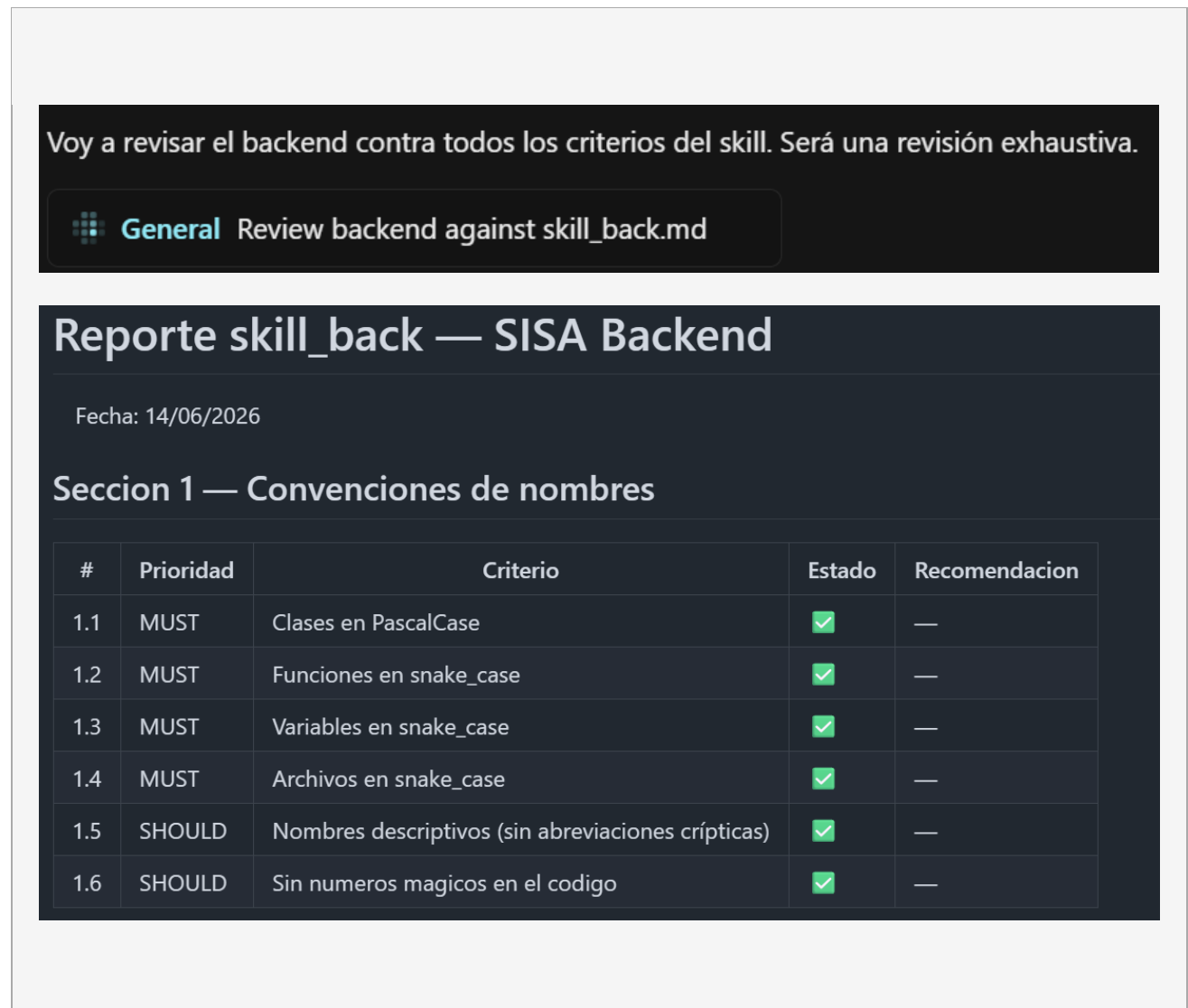
El código fue corregido previamente a la ejecución de las skills para garantizar que todos los criterios fueran cumplidos. Los resultados obtenidos muestran conformidad total con las buenas prácticas definidas, reflejada en el reporte con indicadores verdes para cada criterio evaluado.

2. Resultados skill_back.md - Backend

La skill del backend evalúa 32 criterios distribuidos en cuatro secciones: convenciones de nombres en Python, reglas de clean code, arquitectura en capas (domain / infra / services / controllers / serializers) y documentación del proyecto.

La skill fue ejecutada sobre la carpeta *src/core/* del repositorio. A continuación, se presentan los resultados obtenidos:

Figura 1. Resultados skill_back.md



Seccion 2 — Clean Code

#	Prioridad	Criterio	Estado	Recomendacion
2.1	MUST	Sin comentarios en el codigo (el codigo debe ser autoexplicativo)	✓	—
2.2	MUST	Sin bloques try/except	✓	—
2.3	MUST	Sin bloques switch (cadenas if/elif largas de mas de 3 ramas)	✓	—
2.4	MUST	Sin ifs o fors anidados (maximo 2 niveles de sangria)	✓	—
2.5	MUST	Funciones con maximo 3 argumentos	✓	—
2.6	MUST	Funciones de maximo 20 lineas	✓	—
2.7	MUST	Cada funcion hace una sola cosa	✓	—
2.8	MUST	Un bloque if/while contiene solo una llamada a funcion	✓	—
2.9	SHOULD	Funciones ordenadas: la que no depende de nada va arriba	✓	—
2.10	SHOULD	Principio KISS (la solucion mas simple posible)	✓	—
2.11	SHOULD	Principio SOLID: cada clase tiene una sola responsabilidad (SRP)	✓	—
2.12	SHOULD	DRY: sin bloques de codigo duplicados entre archivos del mismo tipo	✓	—

Seccion 3 — Arquitectura en capas

#	Prioridad	Criterio	Estado	Recomendacion
3.1	MUST	Todos los modelos en domain/ tienen managed = False	✓	—
3.2	MUST	Ningun modelo en domain/ tiene logica de negocio (solo campos y Meta)	✓	—
3.3	MUST	Solo infra/ accede a Model.objects.* (ORM)	✓	—
3.4	MUST	Los services/ reciben el repositorio por inyeccion en init	✓	—
3.5	MUST	Los controllers/ no tienen logica de negocio ni consultas a DB	✓	—
3.6	MUST	Los controllers/ no importan nada de infra/ ni de domain/ directamente	✓	—
3.7	MUST	Los serializers/ solo importan de domain/	✓	—
3.8	MUST	Los tests/ usan MagicMock para reemplazar el repositorio (sin DB real)	✓	—
3.9	SHOULD	Los ForeignKey en domain/ usan referencia en string 'core.Modelo'	✓	—
3.10	SHOULD	Las tablas con PK compuesta tienen unique_together en el Meta	✓	—

Seccion 4 — Documentacion y estilo

#	Prioridad	Criterio	Estado	Recomendacion
4.1	SHOULD	Los archivos <code>init.py</code> de cada carpeta estan presentes	✓	—
4.2	SHOULD	El archivo <code>pytest.ini</code> existe y tiene <code>DJANGO_SETTINGS_MODULE</code> configurado	✓	—
4.3	SHOULD	El archivo <code>.env</code> existe en la raiz del backend y esta listado en <code>.gitignore</code>	✓	—
4.4	SHOULD	<code>requirements.txt</code> incluye las dependencias de testing (<code>pytest</code> , <code>pytest-django</code> , <code>coverage</code> , <code>flake8</code>)	✓	—

Resumen

- ✓ Cumple: 32 criterios
- ⚠ Parcial: 0 criterios
- ✗ No cumple: 0 criterios

¡El proyecto cumple con todas las buenas practicas definidas!

3. Resultados skill_front.md - Frontend

La skill del frontend evalúa 38 criterios distribuidos en cinco secciones: convenciones de nombres en Vue y TypeScript, reglas de clean code, arquitectura en capas (views / components / stores / services / router / types), tipado con TypeScript y configuración del proyecto.

La skill fue ejecutada sobre la carpeta *src/* del repositorio del frontend. A continuación, se presentan los resultados obtenidos:

Figura 2. Resultados skill_front.md

Reporte skill_front — SISA Frontend

Fecha: 2026-06-14

Seccion 1 — Convenciones de nombres

#	Prioridad	Criterio	Estado	Recomendacion
1.1	MUST	Componentes en PascalCase	✓	—
1.2	MUST	Servicios y stores en camelCase	✓	—
1.3	MUST	Interfaces TypeScript en PascalCase	✓	—
1.4	MUST	Variables y funciones en camelCase	✓	—
1.5	MUST	Stores de Pinia con prefijo use y sufijo Store	✓	—
1.6	SHOULD	Nombres descriptivos sin abreviaciones cripticas	✓	—
1.7	SHOULD	Sin numeros magicos	✓	—

Seccion 2 — Clean Code

#	Prioridad	Criterio	Estado	Recomendacion
2.1	MUST	Sin comentarios en el codigo	✓	—
2.2	MUST	Sin bloques try/catch en stores o services	✓	—
2.3	MUST	Sin ifs o ternarios anidados de mas de 2 niveles	✓	—
2.4	MUST	Funciones con maximo 3 argumentos	✓	—
2.5	MUST	Funciones de maximo 20 lineas	✓	—
2.6	MUST	Cada funcion hace una sola cosa	✓	—
2.7	MUST	Sin JSDoc en el codigo	✓	—
2.8	SHOULD	Principio KISS	✓	—
2.9	SHOULD	DRY: sin logica duplicada entre vistas	✓	—
2.10	SHOULD	Principio SRP: cada archivo una sola responsabilidad	✓	—

Seccion 3 — Arquitectura en capas Vue

#	Prioridad	Criterio	Estado	Recomendacion
3.1	MUST	Vistas no importan directamente de services/	✓	—
3.2	MUST	Solo los stores importan de services/	✓	—
3.3	MUST	Stores usan defineStore de Pinia	✓	—
3.4	MUST	Router usa guardias de navegacion (beforeEach)	✓	—
3.5	MUST	Unico cliente HTTP centralizado en services/	✓	—
3.6	MUST	Tipos e interfaces en types/	✓	—
3.7	MUST	Constantes globales en types/ o constants/	✓	—
3.8	SHOULD	Componentes reutilizables sin logica de negocio	✓	—
3.9	SHOULD	Cliente HTTP con interceptor 401	✓	—
3.10	SHOULD	localStorage solo dentro de stores/	✓	—

Seccion 4 — TypeScript y tipado

#	Prioridad	Criterio	Estado	Recomendacion
4.1	MUST	Componentes con script setup lang="ts"	✓	—
4.2	MUST	Props tipadas con defineProps	✓	—
4.3	MUST	Emits tipados con defineEmits	✓	—
4.4	SHOULD	Variables reactivas con tipo explicito	✓	—
4.5	SHOULD	No se usa any como tipo	✓	—

Seccion 5 — Estructura y configuracion

#	Prioridad	Criterio	Estado	Recomendacion
5.1	MUST	Existe vite.config.ts en la raiz	✓	—
5.2	MUST	Existe tsconfig.json en la raiz	✓	—
5.3	MUST	Existe tailwind.config.js en la raiz	✓	—
5.4	MUST	package.json incluye dependencias principales	✓	—
5.5	SHOULD	Existe .env con VITE_API_URL	✓	—
5.6	SHOULD	baseUrl usa variable de entorno	✓	—

Resumen

- ✓ Cumple: 38 criterios
- ⚠ Parcial: 0 criterios
- ✗ No cumple: 0 criterios

¡El proyecto cumple con todas las buenas practicas definidas!